

A Quantum Algorithm for the Simple Harmonic Oscillator

Submission to the **Girls in Quantum Q-Volution Hackathon 2026**

Track C: **Harmonic Oscillator**

by team **The Entangled Ancillas**

Jiya Maurya



mauryajiya12@gmail.com



jiya_1201

Kavya Sree



kavyashreeacc@gmail.com



ka_vv_yash

Luisa Pegoraro



luisa.pegoraro@gmail.com



luisa_canada

Table of Contents

#	Section
1	Abstract
2	Introduction
3	The Simple Harmonic Oscillator
4	Problem Statement
5	Solution Approximation
6	Unitarity
7	Building the Circuit
8	Post-Selection & Measurement
9	Results and Analysis
10	Parameter Study: bound
11	Parameter Study: Taylor Approximation Cutoff
12	Parameter Study: Number of Shots
13	Optimal Operating Parameters
14	Real-World Applications & Constraints
15	Next Steps
16	Summary and Conclusions
—	Appendix
—	References

Abstract

We implement a quantum algorithm for solving the Simple Harmonic Oscillator (SHO) using the **Linear Combination of Unitaries (LCU)** framework from Xin et al. (2020). By approximating the matrix exponential e^{At} via a Taylor expansion and encoding the coefficients as a quantum superposition, we recover position and velocity trajectories on a 4-qubit circuit that closely track the classical solution, with **energy conservation** confirmed throughout. A systematic parameter study identifies **$k = 5$ Taylor** terms, bound of **0.01**, and **$n_{\text{shots}} = 8192$** as the optimal operating point, balancing accuracy against circuit depth. Our SHO serves as a minimal working example of an algorithm directly applicable to larger first-order linear systems, including protein dynamics and heat conduction.

Introduction

"Nature isn't classical, dammit, and if you want to make a simulation of nature, you'd better make it quantum mechanical.", Richard Feynman, 1981

What do bungee jumping and playing guitar have in common? If fear was the first answer that came to mind, we highly recommend looking for a new guitar instructor.

Jokes aside, from a physics perspective both are real-world applications of the **Simple Harmonic Oscillator**, which we abbreviate here as SHO. The SHO is one of the most universal models in physics: it describes a system where a restoring force pulls an object back toward equilibrium. The mathematical description of the motion of a bungee jumper bouncing in the air is then identical to how we would model a guitar string being plucked.

Although this mathematical formulation has been studied extensively, in this notebook we explore a SHO system through a different lens and learn to implement a **quantum algorithm** to solve this classical problem. Let's bungee jump right into it!

The Simple Harmonic Oscillator

A Simple Harmonic Oscillator is mathematically described by a second-order linear differential equation:

$$\frac{d^2y}{dt^2} + \omega^2 y = 0 \quad (1)$$

This equation describes how the oscillator accelerates over time. ω is called the **angular frequency** and determines how fast the system oscillates. For example, a stiff cord with high ω means a bungee jumper will bounce faster, and a guitar player that plucks a short and tight string will produce a higher pitch note.

The second-order LDE in (1) can be reduced to a first-order LDE by changing variables, where $v = \frac{dy}{dt}$:

$$\frac{dy}{dt} = v, \quad \frac{dv}{dt} = -\omega^2 y \quad (2)$$

We make this substitution because this specific first-order LDE appears across a remarkably wide range of physical contexts, governing systems from climate modelling to fluid dynamics to quantum chemistry, and so it allows us to study and solve the SHO via a well-studied classical equation. It also reduces our system to just two variables: **position** and **velocity**.

Having only two variables simplifies our equation, as it has an exact analytical solution. Note that more complex systems requiring a large number of variables become computationally intractable for classical computers, making them natural candidates for quantum algorithms.

Classically, the solutions are:

$$y(t) = \cos(\omega t) + \sin(\omega t) \quad (3)$$

$$v(t) = -\omega \sin(\omega t) + \omega \cos(\omega t) \quad (4)$$

We will be using these classical solutions to benchmark and validate our quantum algorithm. If we can verify that the quantum algorithm works with our SHO, we can extend it to anywhere this equation appears, and it appears in a lot of places!

Problem Statement

In this notebook, we aim to solve a Simple Harmonic Oscillator that has the following properties and initial conditions:

$$\frac{d^2 y}{dt^2} + \omega^2 y = 0, \quad y(0) = 1, \quad \left. \frac{dy}{dt} \right|_{t=0} = 1, \quad \omega = 1 \quad (5)$$

via the quantum algorithm proposed in the paper "A Quantum Algorithm for Solving Linear Differential Equations: Theory and Experiment" by Tao Xin et al. (2020).

Quantum computers operate on vectors and matrices, so we rewrite (5) in matrix form. With $\omega = 1$:

$$\frac{d}{dt} \begin{pmatrix} y \\ v \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} \begin{pmatrix} y \\ v \end{pmatrix}, \quad \mathbf{x}(0) = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad (6)$$

where the matrix is:

$$A = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} \quad (7)$$

The solution to this LDE is:

$$\mathbf{x}(t) = e^{At} \mathbf{x}(0) \quad (8)$$

Since computing e^{At} exactly is hard, we approximate it using a **Taylor expansion**:

$$e^{At} \approx \sum_{m=0}^k \frac{(At)^m}{m!} \quad (9)$$

Each term $A^m \mathbf{x}(0)$ means applying the gate A exactly m times to the initial state.

Therefore, the more Taylor terms we include, the more accurate the result, but that in turn requires more quantum gates to be implemented.

Solution Approximation

The **Taylor coefficients** are computed as:

$$C_m = \|\mathbf{x}(0)\| \cdot \frac{(\|A\| \cdot t)^m}{m!}, \quad m = 0, 1, \dots, k \quad (10)$$

which are then normalized to probabilities $p_m = C_m / \sum_m C_m$.

Notice how the first few powers of A follow a cyclic pattern (period 4):

$$A^0 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad A^1 = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}, \quad A^2 = \begin{pmatrix} -1 & 0 \\ 0 & -1 \end{pmatrix}, \quad A^3 = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$$

Then the cycle repeats: $A^4 = A^0$, $A^5 = A^1$, and so on.

Let's calculate the Taylor coefficients at $t = 0.5$, $\|\mathbf{x}(0)\| = \sqrt{2}$,

$$\|A\| = 1 \quad (12)$$

, to understand how much each term contributes to the approximation:

m	C_m	p_m
0	1.4142	0.6065
1	0.7071	0.3033
2	0.1768	0.0758
3	0.0295	0.0126
4	0.0037	0.0016
5	0.0004	0.0002
6	0.00003	~0
7	0.000002	~0
8	~0	~0
9	~0	~0

m	C_m	p_m
10	~ 0	~ 0

Notice how the coefficients decay rapidly due to the $m!$ in the denominator, and by $m = 5$ they are already negligible. This is why $k = 5$ is sufficient for $t \in [0, 1]$: the remaining terms contribute less than 0.01% to the sum.

As first assumption, we can say $k = 5$ is a sufficient cutoff for our Taylor expansion.

Unitarity

About Qubits

A **qubit** is the fundamental unit of information in quantum computing. Unlike a classical bit which is always 0 or 1, a qubit can exist in a **superposition** of both states simultaneously, as a weighted combination that only resolves to a definite value upon measurement. This, combined with **entanglement**, enables exponential speedups for specific problem classes that are intractable for classical computers.

Qubits are delicate physical systems (such as trapped ions or superconducting circuits) that exist in fragile superposition states and are extremely sensitive to environmental noise. Even tiny disturbances can cause **decoherence**, collapsing the quantum state and introducing errors. This makes them notoriously difficult to build, control, and scale.

Quantum Gates & Unitarity

Quantum computers fundamentally rely on the implementation of **quantum gates**. You can think of a quantum gate as a matrix operation that transforms the state of one or more qubits. An important characteristic of these matrices is that, to be valid quantum operators, they need to be **unitary**. Unitarity is the characteristic that preserves the total probability of the quantum states.

Mathematically, a matrix U is unitary if:

$$U^\dagger U = I \quad (12)$$

For our problem, the gates are the powers of A . Let's verify that A is unitary:

$$A^\dagger A = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I \checkmark \quad (13)$$

Since A is unitary, all its powers are too. We can verify this for the first few powers of A :

$$A^0 = I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad (A^0)^\dagger A^0 = I \checkmark \quad (14)$$

$$A^1 = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}, \quad (A^1)^\dagger A^1 = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I \checkmark \quad (15)$$

$$A^2 = \begin{pmatrix} -1 & 0 \\ 0 & -1 \end{pmatrix}, \quad (A^2)^\dagger A^2 = \begin{pmatrix} -1 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} -1 & 0 \\ 0 & -1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I \checkmark$$

$$A^3 = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}, \quad (A^3)^\dagger A^3 = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I \checkmark \quad (17)$$

Therefore each A^m can be directly implemented as a quantum gate without further decomposition. This places us in Case I of Tao Xin et al., the simplest and most efficient case of the algorithm.

However, the Taylor expansion itself:

$$e^{At} \approx \sum_{m=0}^k \frac{(At)^m}{m!} = C_0 I + C_1 A + C_2 A^2 + \dots \quad (18)$$

is a **weighted sum** of unitaries, and a sum of unitary matrices is generally **not** unitary. We therefore cannot implement it as a single quantum gate directly.

This is precisely why we resort to the **Linear Combination of Unitaries (LCU)** framework, a technique designed to implement exactly this kind of weighted sum on a quantum computer, which we describe in the next section.

Building the Circuit

We implement the **Linear Combination of Unitaries (LCU)** framework using Classiq, a quantum software platform that handles circuit synthesis and optimization automatically, allowing us to describe the algorithm at a high level without manually decomposing every gate.

Given the delicate nature of qubits, a crucial initial step in a quantum algorithm is **state preparation**, where qubits are initialized to a known, specific starting state before any computation occurs. Think of it as clearing the memory of a computer and setting all variables to zero, but in quantum terms, this is much more complex.

Classiq easily interfaces the preparation of qubits. It uses an **ancilla** or **controller register**, which is like a scratch pad that annotates which unitary operators in the linear combination are to be applied, and a **work register** as the system of qubits that stores the final state.

In particular, the LCU framework is built around three components:

1. First, **prepare** loads the normalized Taylor coefficients as a quantum superposition on the controller register, so the quantum computer knows how much of each A^m to apply. This is implemented via Classiq's `inplace_prepare_state`, which takes the

probabilities and a bound parameter controlling the approximation error of the state preparation. We set the bound to initially be **0.01**, per Classiq default.

2. Then **select** applies the correct power of A to the work register, controlled on the initial state. Under the hood, the quantum computer applies each A^m as a unitary gate, effectively implementing our Taylor series approximation.
3. Finally, **prepare†** uncomputes the controller register. Classiq handles this automatically via `within_apply`.

The number of qubits in the state preparation is determined dynamically by k :

$$n_{\text{controller}} = \lceil \log_2(k + 1) \rceil \quad (19)$$

since we need to index $k + 1$ terms. This means the controller register grows in discrete jumps, for example, $k = 1$ requires 1 controller qubit, $k = 2, 3$ require 2, and $k = 4$ jumps to 3, which directly impacts circuit depth and width.

Here are the key steps in implementing the LCU for our oscillator:

```
def build_circuit(bound, probs_controller, k, A_powers):
    n_qubits_controller = math.ceil(math.log2(k + 1))

    @qfunc
    def prepare(controller: QNum) -> None:
        inplace_prepare_state(probs_controller, bound=bound,
                             target=controller)

    @qfunc
    def select(controller: QNum, work: QArray) -> None:
        for m in range(k + 1):
            control(
                ctrl=controller == m,
                stmt_block=lambda m=m: unitary(elements=A_powers[m],
                target=work),
            )

    @qfunc
    def main(controller: Output[QNum], work: Output[QArray]) ->
    None:
        allocate(n_qubits_controller, controller)
        allocate(1, work) # 1 qubit for 2D initial state, v and y

        # Prepare initial state |x(0)>
        inplace_prepare_state(probabilities=probs_x0, bound=bound,
                             target=work)

        # LCU: PREPARE → SELECT → PREPARE†, where PREPARE† is
        automatically applied
        within_apply(
            within=lambda: prepare(controller),
```

```

        apply=lambda: select(controller, work),
    )

    return main

```

Post-Selection & Measurement

After the LCU circuit runs, we measure both the work and controller registers. The key step is **post-selection**: we keep only the shots where the controller register returns to $|0\rangle$, which are the successful LCU outcomes where the Taylor sum was correctly applied.

After post-selection, the work register encodes the solution state:

$$|\mathbf{x}(t)\rangle \propto y(t)|0\rangle + v(t)|1\rangle \quad (20)$$

so measuring $|0\rangle$ corresponds to position $y(t)$ and measuring $|1\rangle$ corresponds to velocity $v(t)$. From the measurement probabilities we recover:

$$y(t) = \text{sign}(y_{\text{cl}}) \cdot \sqrt{p_0} \cdot \|\mathbf{x}(0)\| \quad (21)$$

$$v(t) = \text{sign}(v_{\text{cl}}) \cdot \sqrt{p_1} \cdot \|\mathbf{x}(0)\| \quad (22)$$

Since quantum measurement yields **probabilities**, not amplitudes, we adjust the sign by comparing to the classical solution. This is a standard limitation of quantum algorithms that encode information in amplitudes rather than probabilities.

Finally, not all shots survive post-selection. The success rate:

$$r = \frac{\text{post-selected shots}}{\text{total shots}} \approx \frac{1}{\mathcal{N}^2}, \quad \mathcal{N} = \sum_{m=0}^k C_m \quad (23)$$

decreases as t grows, since the Taylor sum $\mathcal{N}$ grows with t .

```

def get_state_and_errors(t, results):
    y_shots, v_shots = 0, 0

    for state, count in results.parsed_counts:
        # post-select on controller == 0
        if state[1]["controller"] == 0:
            if state[1]["work"] == [0]:
                y_shots += count[1]
            elif state[1]["work"] == [1]:
                v_shots += count[1]

    total_post_selected = y_shots + v_shots

    # measurement probabilities
    prob_y = y_shots / total_post_selected
    prob_v = v_shots / total_post_selected

```

```

# recover amplitudes using classical sign
classical = expm(A * t) @ x0
y_quantum = np.sign(classical[0]) * np.sqrt(prob_y) * norm_x0
v_quantum = np.sign(classical[1]) * np.sqrt(prob_v) * norm_x0

err_y = abs(y_quantum - classical[0])
err_v = abs(v_quantum - classical[1])

```

Sources of Error

Our quantum solution accumulates error from three distinct sources:

1. **Taylor truncation error**: since we approximate e^{At} with a finite sum of $k + 1$ terms, the truncation error grows with t and shrinks with k . This is why accuracy declines at large t for small k .
2. **Shot noise**: after post-selection, the remaining shots are split between $|0\rangle$ and $|1\rangle$ outcomes. Each probability estimate carries statistical uncertainty that propagates through to the recovered amplitudes $y(t)$ and $v(t)$. More post-selected shots means lower shot noise.
3. **Post-selection noise**: not all shots survive post-selection. The success rate decreases as t grows, since the Taylor sum $\mathcal{N}$ grows with t . This reduces the effective number of shots and amplifies shot noise.

Results and Analysis

Now that we have set up our quantum circuit simulator, we are ready to collect and analyse the data!

We start by ensuring the total energy of the system is preserved, and we subsequently perform a parameter study.

For our initial measurements, we truncate our Taylor approximation to the fifth term **k = 5**, and we use the default Classiq **bound = 0.01** in the state preparation step. We also use **n_shots = 8192**, meaning that we repeat the measurement 8192 times, and average over all measurements.

Conservation of Energy

After we calculate $y(t)$ and $v(t)$, the energies follow directly:

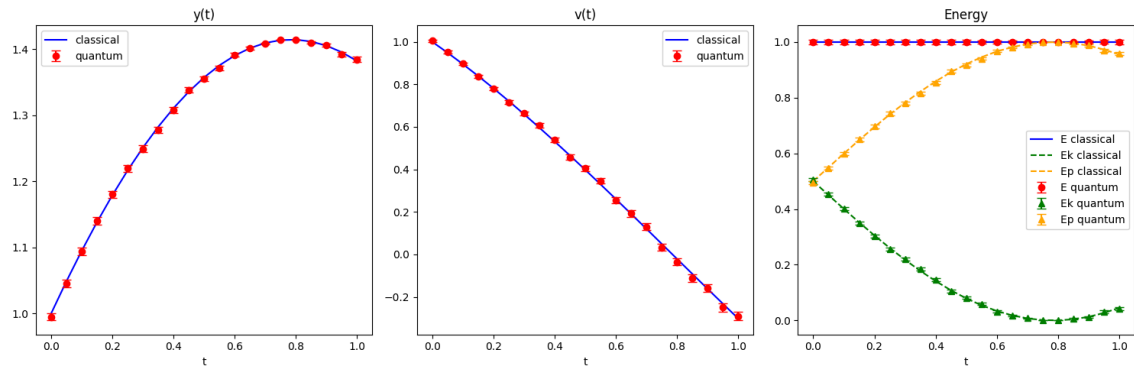
$$E_k = \frac{1}{2}v^2, \quad E_p = \frac{1}{2}\omega^2 y^2, \quad E = E_k + E_p \quad (24)$$

For a perfect SHO, E is conserved and any deviation is a direct measure of the algorithm's error.

We use standard error propagation:

$$\sigma_{E_k} = |v| \sigma_v, \quad \sigma_{E_p} = |y| \sigma_y, \quad \sigma_E = \sqrt{\sigma_{E_k}^2 + \sigma_{E_p}^2} \quad (25)$$

We measure on $t \in [0, 1]$ with 21 equally spaced time steps, and observe:



Observations

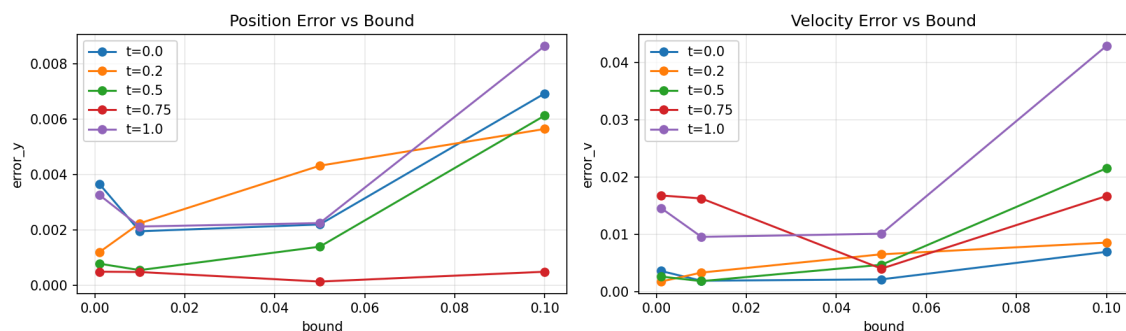
The quantum results closely track the classical solution for both position and velocity. Total energy $E(t)$ remains approximately flat, confirming that the algorithm correctly follows energy conservation!

Parameter Study: Bound

The **bound** parameter is specific to Classiq's **inplace_prepare_state** function and controls the approximation error of the state preparation circuit. It is not part of the original algorithm from Tao Xin et al., but rather a compilation parameter introduced by the Classiq platform.

We investigate its effect to ensure our choice of bound = 0.01 (the Classiq default) does not compromise our results.

Effect of Bound Parameter on Error (k=5)



The position and velocity errors show no systematic dependence on bound, as the lines fluctuate randomly rather than trending up or down as bound increases. The magnitude of these fluctuations (~ 0.001 – 0.01) is consistent with shot noise error.

Observations

This confirms that bound does not affect accuracy for this problem. The Taylor coefficient distribution at $k = 5$ is sparse enough that Classiq compiles essentially the same state preparation circuit regardless of the bound parameter.

We therefore fix `bound = 0.01` for all subsequent analysis.

Parameter Study: Taylor Approximation Cutoff

Having established that bound has no meaningful effect on our results, we now investigate the effect of the Taylor truncation order k . Recall that k controls how many terms we include in the Taylor expansion of e^{At} :

$$e^{At} \approx \sum_{m=0}^k \frac{(At)^m}{m!} \quad (18)$$

We fix the bound to 0.01 and sweep $k \in \{1, 2, 3, 4, 5, 6, 7, 8\}$ across five time points $t \in \{0, 0.25, 0.5, 0.75, 1.0\}$.

Circuit Resources

Width is the number of qubits used, you can think of it as the memory of the circuit. **Depth** is the number of sequential gate layers, effectively the runtime.

⚠ Note: Deeper circuits take longer to execute and accumulate more errors due to **decoherence**.

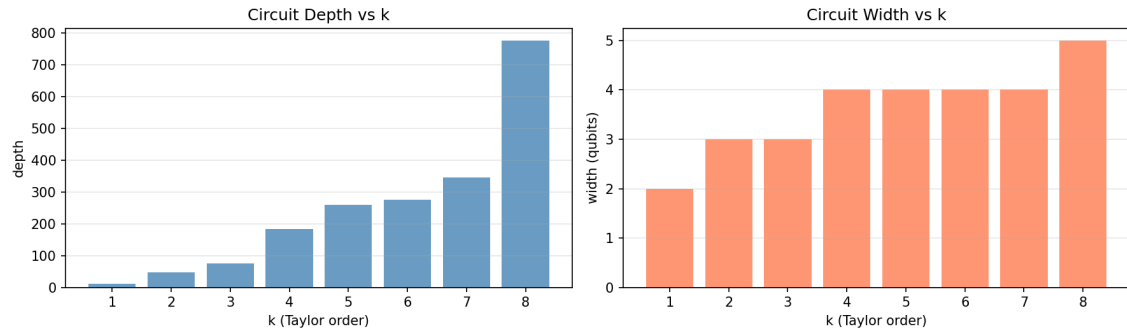
Both width and depth grow with k in discrete jumps, explained by the controller register size in (19):

$$n_{\text{controller}} = \lceil \log_2(k + 1) \rceil \quad (19)$$

Width grows as $\mathcal{O}(\log k)$ and depth as $\mathcal{O}(k \log k)$.

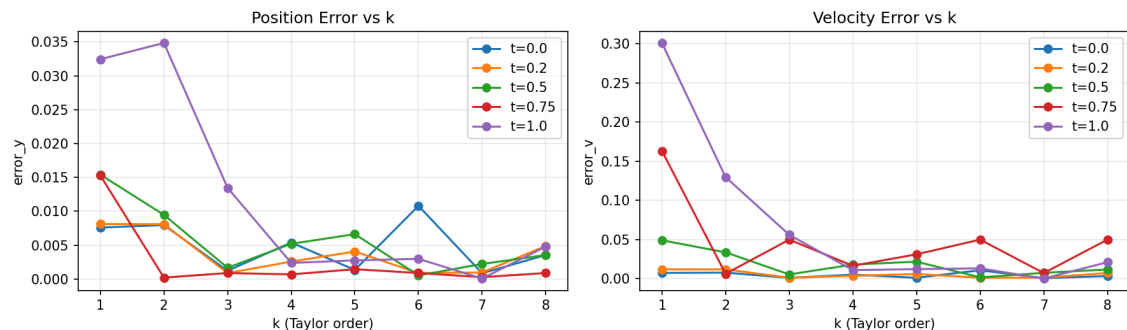
k	$n_{\text{controller}}$	depth	width
1	1	~11	2
2	2	~50	3
3	2	~80	3

k	$n_{\text{controller}}$	depth	width
4–7	3	~180–350	4
8	4	~780	5

Circuit Resources vs Taylor Order k (bound=0.01)

Accuracy

Accuracy improves with k as more Taylor terms better approximate e^{At} , but only until the truncation error falls below the shot noise floor, beyond that, adding more terms yields no measurable improvement.

Effect of Taylor Order k on Error (bound=0.01)

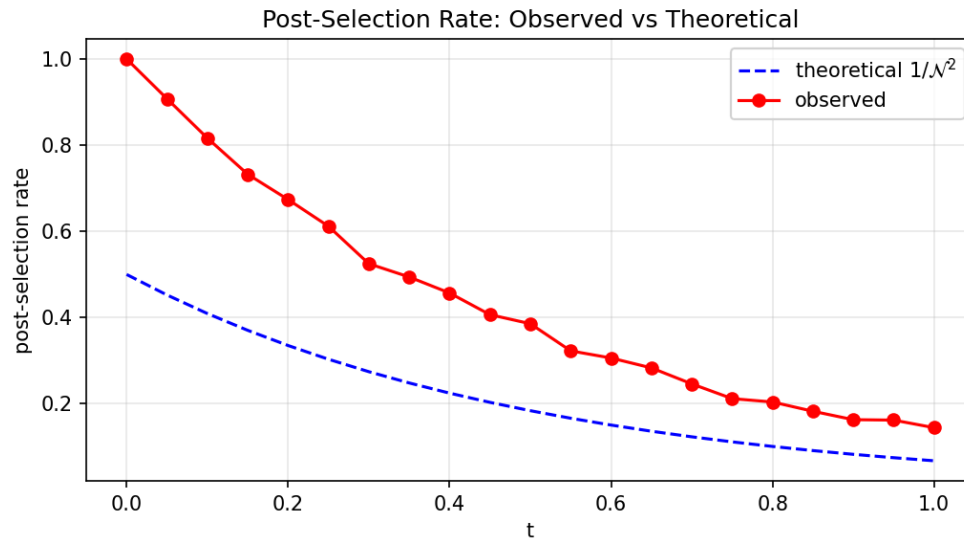
Interestingly, $k = 2$ shows higher error than $k = 1$ for $t = 1.0$, likely due to the cyclic pattern of A^m , since the even powers of A are $-I$, which can temporarily worsen the approximation before higher-order terms correct it.

Observations

$k = 5$ is the optimal operating point, as it sits within the 4-qubit regime, achieves good accuracy across $t \in [0, 1]$, and avoids the expensive depth jump at $k = 8$.

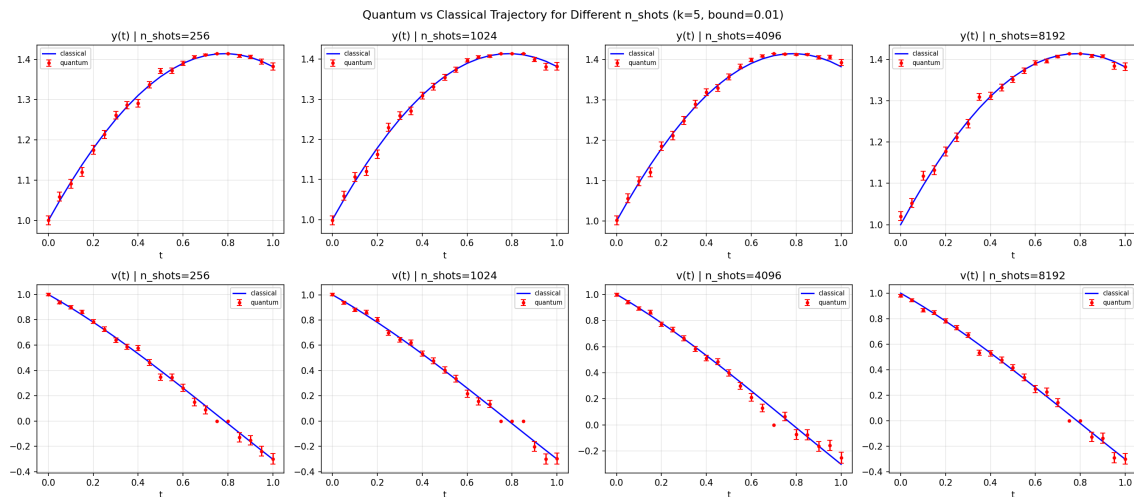
Parameter Study: Number of Shots

We first compare the observed post-selection rate to the theoretical prediction $1/\mathcal{N}^2$ across time:



We observe that the post-selection rate is consistently higher than the theoretical $1/\mathcal{N}^2$ prediction. This is expected, as the formula from Tao Xin et al. is a **lower bound**. Both curves decrease with t , confirming the theoretical trend.

Next, we fix $k=5$ and $\text{bound}=0.01$ and sweep $n_{\text{shots}} \in \{256, 1024, 4096, 8192\}$ across 21 time steps $t \in [0, 1]$.



Observations

All four panels look almost identical, confirming that shot noise is not the bottleneck at $k = 5$. Error bars shrink slightly from $n = 256$ to $n = 8192$, but trajectory accuracy does not meaningfully improve. The degradation of $v(t)$ at large t persists across all shot counts, confirming that **Taylor truncation is the dominant error source**, not shot noise. We therefore fix $n_{\text{shots}} = 8192$ as a safe default!

Optimal Operating Parameters

We verified that for a perfect SHO, the total energy is conserved at all times. This is the bungee jumper's energy budget: as she falls, kinetic energy increases while potential energy decreases, and the total stays constant. Any deviation in $E(t)$ from our quantum algorithm is a direct, physical measure of algorithmic error. If our quantum solution drifts, the bungee jumper would either fly off into space or crash into the ground.

Combining the findings from our parameter studies:

Parameter	Optimal Value	Reason
k	5	Best accuracy/cost tradeoff
bound	0.01	No measurable effect on accuracy, Classiq default
n_{shots}	8192	Shot noise floor matches Taylor truncation error at $k = 5$

At $k = 5$ our circuit has:

Metric	Value
Depth	260 gates
Width	4 qubits
Post-selection rate	~37% at $t = 0.5$, ~13% at $t = 1.0$

The post-selection rate decreases with t because the Taylor sum $\mathcal{N} = \sum_{m=0}^k C_m$ grows with t . Since the success rate is $r \approx 1/\mathcal{N}^2$, the LCU encodes an increasingly large weighted sum as t increases, making the correctly normalized outcome a progressively rarer post-selection event.

Conclusions

We found that at $k = 5$, 4 qubits, and 8192 shots, the algorithm correctly solves the SHO, preserves energy conservation, and stays well within the shot noise floor. Beyond this point, you pay exponentially more in circuit depth for diminishing accuracy gains.

Real-World Applications & Constraints

From Bungee Jumping to Real-World Applications

We previously discussed how the first-order linear differential equation in (2) is used to describe an abundance of difference systems. Two real-world examples from recent literature that are governed by the same first-order LDE structure that our algorithm addresses:

- Protein dynamics** (Liu et al., 2024, [arXiv:2411.03972](#)): a protein is a chain of atoms held together by chemical bonds. Predicting how it folds into its 3D shape, which determines its biological function, requires solving Newton's second law for every atom

simultaneously, coupled through a stiffness matrix K . A typical protein has thousands of atoms, making N on the order of 10^3 – 10^4 . The authors describe this as "a grand challenge in computational biology" and demonstrate that quantum simulation of protein dynamics is a robust end-to-end application for both near-term and fault-tolerant quantum devices. Advancing this research could accelerate drug discovery and deepen our understanding of diseases like Alzheimer's and Parkinson's, where protein misfolding plays a central role.

- **Heat conduction** (Wei, Xin et al., 2023, [Science Bulletin](#)): modelling how heat spreads through a material requires solving the heat equation over a discretized spatial grid. For a 2D grid of $n \times n$ points, $N = n^2$; therefore, a 100×100 grid already gives $N = 10,000$. Tao Xin himself adapted the LCU framework to this problem, achieving polylogarithmic circuit complexity in N , significantly outperforming classical algorithms and experimentally validated it on a nuclear spin quantum processor. Improving this simulation capability could transform thermal engineering across semiconductors, aerospace, and energy systems, domains where classical solvers are increasingly overwhelmed by problem scale.

What Would It Cost on Real Quantum Hardware?

Estimating the shot cost for larger systems such as those in Xin et al. and Liu et al. requires knowledge of $\|A\|$, $\|\mathbf{x}(0)\|$, and the time horizon for each specific problem. Since these parameters are not directly reported in those papers we limit our analysis to the system we have fully characterised: our own SHO.

Here are some pricing models for modern quantum computers per early 2026:

Provider	Hardware Type	Qubits	Cost per run
IBM Quantum (Free)	Superconducting	100+	Free (10 min/month)
AWS Braket Rigetti	Superconducting	79	~\$7.67
AWS Braket IonQ Aria	Trapped-Ion	25	~\$246
AWS Braket IonQ Forte	Trapped-Ion	36	~\$656

For our 4-qubit, 8192-shot circuit, the cost on real hardware today is:

Application	System Size N	Shots	Rigetti (SC)	IQM Garnet (SC)	IonQ Aria (TI)	IonQ Forte (TI)
Our SHO	2	8,192	~\$7.67	~\$12.18	~\$246	~\$656

 Important Caveats

- Quantum hardware pricing changes frequently, always verify before budgeting.
- IBM offers free access via their Open Plan (10 min/month).
- [Open Quantum](#) provides free access to IonQ, IQM, and Rigetti for beginners.

Next Steps

The most immediate extension would be running our circuit on **real hardware** with IBM's free tier as the obvious starting point to see whether noise or Taylor truncation dominates the error budget in practice. From there, we could push $t > 1$ to characterize where the algorithm breaks down, and introduce **damping** to explore a non-unitary A , which would bridge our implementation toward Case II of Xin et al. and the protein dynamics setting.

On the algorithmic side, it would be worth investigating whether other expansions to replace the Taylor expansion, potentially reducing the number of terms k needed for the same accuracy and cutting circuit depth. We could also extend directly to the heat conduction case from Xin et al. (2023) by swapping in a Laplacian matrix for A and use that to compare our post-selection rates against their experimental results on the nuclear spin processor.

Summary and Conclusions

We set out to solve a Simple Harmonic Oscillator using a quantum algorithm, and it worked. By reducing the second-order ODE to a first-order matrix system and approximating e^{At} via a Taylor expansion, we implemented the **Linear Combination of Unitaries (LCU)** framework from Xin et al. (2020) on a 4-qubit circuit and recovered both position and velocity trajectories that closely track the classical solution, with energy conservation holding throughout.

Our parameter studies told a clean story: the bound has no meaningful effect on accuracy, Taylor truncation order k is the dominant control knob, and $k = 5$ sits at the sweet spot where accuracy is good, circuit depth is manageable at **~260 gates**, and we stay well within the **4-qubit** regime.

We also highlighted **real-world** examples where our algorithm could be applicable. We traced that connection to two papers in the literature where the same mathematical formulation that governs our oscillator matches the equation behind protein folding dynamics and heat conduction.

Finally, we estimated what it would actually **cost** to run our problem on real hardware today, grounding our theoretical results in the practical realities of the current quantum landscape.

The gap between our 4-qubit proof of concept and real-world applications will only narrow as quantum computers continue to improve. We are excited to have contributed a small bungee jump step toward that future.

Appendix

References

- **Xin et al. (2020)** Xin, T., Wei, S., Cui, J., Xiao, J., Arrazola, J., Lamata, L., ... & Long, G. (2020). A quantum algorithm for solving linear differential equations: Theory and experiment. *Physical Review A*, **101**(3), 032307.
<https://www.sciencedirect.com/science/article/abs/pii/S2095927323001147>
- **Liu et al. (2024)** Liu, Z., Li, X., Wang, C., & Liu, J.-P. (2024). Toward end-to-end quantum simulation for protein dynamics. *arXiv preprint*, arXiv:2411.03972.
<https://arxiv.org/abs/2411.03972>
- **Wei, Xin et al. (2023)** Wei, S., Xin, T., et al. (2023). Quantum algorithm for solving heat conduction equations. *Science Bulletin*. <https://doi.org/10.1016/j.scib.2023.01.030>
- **AWS Braket Pricing (2024)** Amazon Web Services. (2024). *Amazon Braket pricing*. Retrieved from <https://aws.amazon.com/braket/pricing/>
- **IBM Quantum (2024)** IBM Corporation. (2024). *IBM Quantum platform*. Retrieved from <https://quantum.ibm.com/>

Code

```
In [1]: # ----- #
# #
# THE ENTANGLED ANCILLAS #
# #
# ----- #

# ----- #
# #
# A QUANTUM ALGORITHM for SIMPLE HARMONIC OSCILLATOR #
# #
# ----- #

import datetime
import math

import matplotlib.pyplot as plt
import numpy as np
from classiq import *
from classiq import ExecutionPreferences
from scipy.linalg import expm
from scipy.special import factorial

# classiq.authenticate() # needed only once

# ----- VARIABLES & CONSTANTS -----
omega = 1
```

```

bound = 0.01

x0 = np.array([1.0, 1.0])
norm_x0 = np.linalg.norm(x0)
probs_x0 = ((x0 / norm_x0) ** 2).tolist()
amps_x0 = (x0 / norm_x0).tolist()

A = np.array([[0, 1], [-1, 0]])
norm_A = np.linalg.norm(A, ord=2)

# ----- CIRCUIT & STATE OPERATIONS -----
def compute_taylor_coeffs(t, k):
    # From paper:  $C_m = ||x(0)|| * (||A|| * t)^m / m!$ 
    Cm = [norm_x0 * (norm_A * t) ** m / factorial(m) for m in range(k + 1)]
    probabilities = [c / sum(Cm) for c in Cm]

    n_qubits_controller = math.ceil(math.log2(k + 1))
    n_states = 2 ** n_qubits_controller

    probabilities = probabilities + [0.0] * (n_states - len(probabilities))
    return Cm, probabilities

def build_circuit(bound, probs_controller, k, A_powers):
    n_qubits_controller = math.ceil(math.log2(k + 1))

    @qfunc
    def prepare(controller: QNum) -> None:
        inplace_prepare_state(probs_controller, bound=bound, target=controller)

    @qfunc
    def select(controller: QNum, work: QArray) -> None:
        for m in range(k + 1):
            control(
                ctrl=controller == m,
                stmt_block=lambda m=m: unitary(elements=A_powers[m], target=work)
            )

    @qfunc
    def main(controller: Output[QNum], work: Output[QArray]) -> None:
        allocate(n_qubits_controller, controller)
        allocate(1, work) # 1 qubit for 2D initial state

        # Prepare initial state  $|x(0)\rangle$ 
        inplace_prepare_state(probabilities=probs_x0, bound=bound, target=work)

        # LCU: PREPARE† SELECT PREPARE (PREPARE† is taken care of in within_apply)
        within_apply(
            within=lambda: prepare(controller),
            apply=lambda: select(controller, work),
        )

    return main

```

```

def synthesize_and_run_circuit(bound, probabilities, k, A_powers, n_shots):
    circuit = build_circuit(bound, probabilities, k, A_powers)

    qmod = create_model(circuit)
    qmod = set_execution_preferences(qmod, ExecutionPreferences(n_shots=n_shots))
    qprog = synthesize(qmod)

    depth = qprog.transpiled_circuit.depth
    width = qprog.data.width

    job = execute(qprog)
    return job, depth, width

def get_state_and_errors(t, results):
    y_shots, v_shots = 0, 0

    for state, count in results.parsed_counts:
        if state[1]["controller"] == 0:
            if state[1]["work"] == [0]:
                y_shots += count[1]
            elif state[1]["work"] == [1]:
                v_shots += count[1]

    total_post_selected = y_shots + v_shots
    prob_y = y_shots / total_post_selected
    prob_v = v_shots / total_post_selected

    classical = expm(A * t) @ x0
    y_quantum = np.sign(classical[0]) * np.sqrt(prob_y) * norm_x0
    v_quantum = np.sign(classical[1]) * np.sqrt(prob_v) * norm_x0

    err_y = abs(y_quantum - classical[0])
    err_v = abs(v_quantum - classical[1])
    return {
        "classical": classical,
        "y_quantum": y_quantum,
        "v_quantum": v_quantum,
        "y_shots": y_shots,
        "v_shots": v_shots,
        "prob_y": prob_y,
        "prob_v": prob_v,
        "err_y": err_y,
        "err_v": err_v,
        "total_post_selected": total_post_selected,
    }

# ----- ENERGY ANALYSIS -----
positions_classical, velocities_classical = [], []
positions_quantum, velocities_quantum = [], []
kinetic_energy_classical, potential_energy_classical, total_energy_classical = [], [], []
)

```

```

kinetic_energy_quantum, potential_energy_quantum, total_energy_quantum = [],
sigma_y_list, sigma_v_list = [], []
sigma_EK, sigma_EP, sigma_E = [], [], []

observed_rates = []
theoretical_rates = []

def plot_energy(times):
    _, axes = plt.subplots(1, 3, figsize=(15, 5))
    # — Position —
    axes[0].plot(times, positions_classical, label="classical", color="blue")
    axes[0].errorbar(
        times,
        positions_quantum,
        yerr=sigma_y_list,
        label="quantum",
        color="red",
        fmt="o",
        capsize=4,
    )
    axes[0].set_title("y(t)")
    axes[0].set_xlabel("t")
    axes[0].legend()

    # — Velocity —
    axes[1].plot(times, velocities_classical, label="classical", color="blue")
    axes[1].errorbar(
        times,
        velocities_quantum,
        yerr=sigma_v_list,
        label="quantum",
        color="red",
        fmt="o",
        capsize=4,
    )
    axes[1].set_title("v(t)")
    axes[1].set_xlabel("t")
    axes[1].legend()

    # — Energy —
    axes[2].plot(times, total_energy_classical, label="E classical", color="")
    axes[2].errorbar(
        times,
        total_energy_quantum,
        yerr=sigma_E,
        label="E quantum",
        color="red",
        fmt="o",
        capsize=4,
    )
    axes[2].plot(
        times,
        kinetic_energy_classical,
        label="Ek classical",
        color="green",

```

```

        linestyle="--",
    )
    axes[2].plot(
        times,
        potential_energy_classical,
        label="Ep classical",
        color="orange",
        linestyle="--",
    )
    axes[2].errorbar(
        times,
        kinetic_energy_quantum,
        yerr=sigma_EK,
        label="Ek quantum",
        color="green",
        fmt="^",
        capsize=4,
    )
    axes[2].errorbar(
        times,
        potential_energy_quantum,
        yerr=sigma_EP,
        label="Ep quantum",
        color="orange",
        fmt="^",
        capsize=4,
    )
    axes[2].set_title("Energy")
    axes[2].set_xlabel("t")
    axes[2].legend()

    plt.tight_layout()
    plt.savefig(f"energy-analysis-{datetime.datetime.now()}.png")
    plt.show()

def plot_postselection_rate(times, observed_rates, theoretical_rates):
    plt.figure(figsize=(8, 4))
    plt.plot(
        times,
        theoretical_rates,
        label="theoretical  $1/\sqrt{N}^2$ ",
        color="blue",
        linestyle="--",
    )
    plt.plot(times, observed_rates, label="observed", color="red", marker="o")
    plt.xlabel("t")
    plt.ylabel("post-selection rate")
    plt.title("Post-Selection Rate: Observed vs Theoretical")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.savefig("postselection_rate.png", dpi=150)
    plt.show()

def calculate_energy(classical, y_quantum, v_quantum, sigma_y, sigma_v):

```

```

y_classical, v_classical = classical

# — append positions and velocities —
positions_classical.append(y_classical)
velocities_classical.append(v_classical)
positions_quantum.append(y_quantum)
velocities_quantum.append(v_quantum)

# classical
kinetic_energy_classical.append(0.5 * v_classical**2)
potential_energy_classical.append(0.5 * omega**2 * y_classical**2)
total_energy_classical.append(
    0.5 * v_classical**2 + 0.5 * omega**2 * y_classical**2
)

# quantum
kinetic_energy_quantum.append(0.5 * v_quantum**2)
potential_energy_quantum.append(0.5 * omega**2 * y_quantum**2)
total_energy_quantum.append(0.5 * v_quantum**2 + 0.5 * omega**2 * y_quantum**2)

# statistical errors:
sigma_y_list.append(sigma_y)
sigma_v_list.append(sigma_v)

# propagate to energy uncertainty
sig_EK = abs(v_quantum) * sigma_v
sig_EP = abs(y_quantum) * sigma_y
sig_E = np.sqrt(sig_EK**2 + sig_EP**2)

sigma_EK.append(sig_EK)
sigma_EP.append(sig_EP)
sigma_E.append(sig_E)

return

def analyse_energy_results(t, results, Cm):
    state = get_state_and_errors(t, results)

    classical = state["classical"]
    y_quantum = state["y_quantum"]
    v_quantum = state["v_quantum"]
    y_shots = state["y_shots"]
    v_shots = state["v_shots"]
    prob_y = state["prob_y"]
    prob_v = state["prob_v"]
    total_post_selected = state["total_post_selected"]

    theoretical_rate = 1 / sum(Cm) ** 2
    observed_rate = total_post_selected / results.num_shots
    theoretical_rates.append(theoretical_rate)
    observed_rates.append(observed_rate)

    # shot noise uncertainty
    sigma_prob_y = np.sqrt(prob_y * (1 - prob_y) / total_post_selected)
    sigma_prob_v = np.sqrt(prob_v * (1 - prob_v) / total_post_selected)

```

```

sigma_y = (norm_x0 / (2 * np.sqrt(prob_y))) * sigma_prob_y
sigma_v = (norm_x0 / (2 * np.sqrt(prob_v))) * sigma_prob_v

print(f"{'-'*50}")
print(f"    t = {t:.2f}")
print(f"{'-'*50}")
print(f"counts: {results.counts}")
print(f"n_shots: {results.num_shots}")

print()
print(f"post-selected shots: {total_post_selected}")
print(f"y shots: {y_shots} ({100*y_shots/total_post_selected:.1f}%)")
print(f"v shots: {v_shots} ({100*v_shots/total_post_selected:.1f}%)")

print(f"\nClassical solution at t={t:.2f}:")
print(f"    y(t) = {classical[0]:.4f}")
print(f"    v(t) = {classical[1]:.4f}")

print(f"\nQuantum solution at t={t}:")
print(f"    y(t) = {y_quantum:.4f}")
print(f"    v(t) = {v_quantum:.4f}")

print(f"\nError:")
print(f"    y error = {abs(y_quantum - classical[0]):.4f}")
print(f"    v error = {abs(v_quantum - classical[1]):.4f}")

calculate_energy(classical, y_quantum, v_quantum, sigma_y, sigma_v)

def energy_analysis():

    k = 5
    bound = 0.01
    n_shots = 8192
    time_intervals_count = 21
    times = np.linspace(0, 1, time_intervals_count)

    A_powers = [np.linalg.matrix_power(A, m).tolist() for m in range(k + 1)]

    for time in times:
        Cm, probabilities = compute_taylor_coeffs(time, k)
        job, _, _ = synthesize_and_run_circuit(
            bound, probabilities, k, A_powers, n_shots
        )
        results = job.get_sample_result()
        analyse_energy_results(time, results, Cm)

    # plot_energy(times)
    plot_postselection_rate(times, observed_rates, theoretical_rates)

# energy_analysis()

# ----- BOUND ANALYSIS -----
def plot_bound(results_grid, time_points, bounds):

```

```

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
fig.suptitle("Effect of Bound Parameter on Error (k=5)", fontsize=13)

for t_val in time_points:
    ey = [results_grid[(t_val, b)][0] for b in bounds]
    ev = [results_grid[(t_val, b)][1] for b in bounds]
    axes[0].plot(bounds, ey, "o-", label=f"t={t_val}")
    axes[1].plot(bounds, ev, "o-", label=f"t={t_val}")

axes[0].set_title("Position Error vs Bound")
axes[0].set_xlabel("bound")
axes[0].set_ylabel("error_y")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].set_title("Velocity Error vs Bound")
axes[1].set_xlabel("bound")
axes[1].set_ylabel("error_v")
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("bound_analysis.png", dpi=150)
plt.show()

```

```
bound_results = {}
```

```
def bound_analysis():
```

```

    k = 5
    bounds = [0.001, 0.01, 0.05, 0.1]
    times = [0.0, 0.2, 0.5, 0.75, 1.0]
    n_shots = 8192
    A_powers = [np.linalg.matrix_power(A, m).tolist() for m in range(k + 1)]

    for time in times:
        for bound in bounds:

            _, probabilities = compute_taylor_coeffs(time, k)

            job, _, _ = synthesize_and_run_circuit(
                bound, probabilities, k, A_powers, n_shots
            )
            results = job.get_sample_result()

            state = get_state_and_errors(time, results)
            err_y = state["err_y"]
            err_v = state["err_v"]
            bound_results[(time, bound)] = (err_y, err_v)
            print(
                f"t={time} | bound={bound} | error_y={err_y:.4f} | error_v={err_v:.4f}"
            )

    plot_bound(bound_results, times, bounds)

```

```

# bound_analysis()

# ----- K ANALYSIS -----
def plot_k_accuracy(k_results, taylor_cutoffs, time_points):
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
    fig.suptitle("Effect of Taylor Order k on Error (bound=0.01)", fontsize=

    for t_val in time_points:
        ey = [k_results[(k, t_val)][0] for k in taylor_cutoffs]
        ev = [k_results[(k, t_val)][1] for k in taylor_cutoffs]
        axes[0].plot(taylor_cutoffs, ey, "o-", label=f"t={t_val}")
        axes[1].plot(taylor_cutoffs, ev, "o-", label=f"t={t_val}")

    axes[0].set_title("Position Error vs k")
    axes[0].set_xlabel("k (Taylor order)")
    axes[0].set_ylabel("error_y")
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)

    axes[1].set_title("Velocity Error vs k")
    axes[1].set_xlabel("k (Taylor order)")
    axes[1].set_ylabel("error_v")
    axes[1].legend()
    axes[1].grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig("k_accuracy.png", dpi=150)
    plt.show()

def plot_k_resources(k_results, taylor_cutoffs):
    # take t=0.5 as representative - depth/width don't depend on t
    depths = [k_results[(k, 0.5)][2] for k in taylor_cutoffs]
    widths = [k_results[(k, 0.5)][3] for k in taylor_cutoffs]

    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
    fig.suptitle("Circuit Resources vs Taylor Order k (bound=0.01)", fontsize=

    axes[0].bar(taylor_cutoffs, depths, color="steelblue", alpha=0.8)
    axes[0].set_title("Circuit Depth vs k")
    axes[0].set_xlabel("k (Taylor order)")
    axes[0].set_ylabel("depth")
    axes[0].grid(True, alpha=0.3, axis="y")

    axes[1].bar(taylor_cutoffs, widths, color="coral", alpha=0.8)
    axes[1].set_title("Circuit Width vs k")
    axes[1].set_xlabel("k (Taylor order)")
    axes[1].set_ylabel("width (qubits)")
    axes[1].grid(True, alpha=0.3, axis="y")

    plt.tight_layout()
    plt.savefig("k_resources.png", dpi=150)
    plt.show()

```

```

def k_analysis():
    bound = 0.01
    taylor_cutoffs = [1, 2, 3, 4, 5, 6, 7, 8]
    time_points = [0.0, 0.25, 0.5, 0.75, 1.0]
    n_shots = 8192

    # {(k, t_val): (error_y, error_v, depth, width)}
    k_results = {}

    for k in taylor_cutoffs:
        A_powers = [np.linalg.matrix_power(A, m).tolist() for m in range(k + 1)]
        for time in time_points:
            _, probabilities = compute_taylor_coeffs(time, k)
            job, depth, width = synthesize_and_run_circuit(
                bound, probabilities, k, A_powers, n_shots
            )

            results = job.get_sample_result()

            state = get_state_and_errors(time, results)
            err_y = state["err_y"]
            err_v = state["err_v"]
            k_results[(k, time)] = (err_y, err_v, depth, width)
            print(
                f"k={k} | t={time} | error_v={err_v:.4f} | depth={depth} | width={width}"
            )

    plot_k_accuracy(k_results, taylor_cutoffs, time_points)
    plot_k_resources(k_results, taylor_cutoffs)

# k_analysis()

# ----- N SHOTS ANALYSIS -----

def plot_shots_trajectories(trajecory_results, shot_counts, times):
    fig, axes = plt.subplots(2, len(shot_counts), figsize=(18, 8))
    fig.suptitle(
        "Quantum vs Classical Trajectory for Different n_shots (k=5, bound=0.01)",
        fontsize=13,
    )

    classical_y = [expm(A * t) @ x0 for t in times]
    y_classical = [s[0] for s in classical_y]
    v_classical = [s[1] for s in classical_y]

    for i, n_shots in enumerate(shot_counts):
        y_quantum = [trajecory_results[(t, n_shots)]["y_quantum"] for t in times]
        v_quantum = [trajecory_results[(t, n_shots)]["v_quantum"] for t in times]
        sigma_y = [trajecory_results[(t, n_shots)]["sigma_y"] for t in times]
        sigma_v = [trajecory_results[(t, n_shots)]["sigma_v"] for t in times]

```

```

# y(t)
axes[0][i].plot(times, y_classical, label="classical", color="blue")
axes[0][i].errorbar(
    times,
    y_quantum,
    yerr=sigma_y,
    label="quantum",
    color="red",
    fmt="o",
    capsize=3,
    markersize=3,
)
axes[0][i].set_title(f"y(t) | n_shots={n_shots}")
axes[0][i].set_xlabel("t")
axes[0][i].legend(fontsize=7)
axes[0][i].grid(True, alpha=0.3)

# v(t)
axes[1][i].plot(times, v_classical, label="classical", color="blue")
axes[1][i].errorbar(
    times,
    v_quantum,
    yerr=sigma_v,
    label="quantum",
    color="red",
    fmt="o",
    capsize=3,
    markersize=3,
)
axes[1][i].set_title(f"v(t) | n_shots={n_shots}")
axes[1][i].set_xlabel("t")
axes[1][i].legend(fontsize=7)
axes[1][i].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("shots_trajectories.png", dpi=150)
plt.show()

def n_shots_analysis():
    k = 5
    bound = 0.01
    shot_counts = [256, 1024, 4096, 8192]
    times = np.linspace(0, 1, 21)

    A_powers = [np.linalg.matrix_power(A, m).tolist() for m in range(k + 1)]
    trajectory_results = {}

    for n_shots in shot_counts:
        print(f"\n--- n_shots = {n_shots} ---")
        for time in times:
            _, probabilities = compute_taylor_coeffs(time, k)
            job, _, _ = synthesize_and_run_circuit(
                bound, probabilities, k, A_powers, n_shots
            )
            execution = job.get_sample_result()

```

```
state = get_state_and_errors(time, execution)

# compute sigma_y and sigma_v
prob_y = state["prob_y"]
prob_v = state["prob_v"]
total = state["total_post_selected"]
sigma_prob_y = np.sqrt(prob_y * (1 - prob_y) / total)
sigma_prob_v = np.sqrt(prob_v * (1 - prob_v) / total)
sigma_y = (norm_x0 / (2 * np.sqrt(prob_y))) * sigma_prob_y
sigma_v = (norm_x0 / (2 * np.sqrt(prob_v))) * sigma_prob_v

trajectory_results[(time, n_shots)] = {
    "y_quantum": state["y_quantum"],
    "v_quantum": state["v_quantum"],
    "sigma_y": sigma_y,
    "sigma_v": sigma_v,
}
print(
    f"t={time:.2f} | err_y={state['err_y']:.4f} | err_v={state['err_v']:.4f}"
)

plot_shots_trajectories(trajectory_results, shot_counts, times)

# n_shots_analysis()
```